One-to-one SkewGRAM : appliquer des embeddings du TAL à l'optimisation de graphes biologiques

Bechir Mady[1], Abderahmane Abderrahmane[1], Mohamedou Yahya Cheikh Mohamed Vall[1], El Moustapha Mohamed El Moustapha[1], Mohamed Lemine Ahmed Sidi[1] et Hafedh Mohamed Babou[2]

[1] Unité de Recherche en Systèmes Intelligents Avancés (URSIA), Institut Supérieur du Numérique (SupNum), Nouakchott, Mauritanie
{22014,22053,25239,22013}@supnum.mr, m-lamine.ahmed-sidi@supnum.mr
[2] École Supérieure Polytechnique (ESP), Nouakchott, Mauritanie
hafedh.mohamed-babou@esp.mr

Résumé. Le problème One-to-one SkewGRAM consiste à trouver un plus long chemin dans un graphe orienté acyclique D dont les sommets induisent un sous-graphe connexe dans un graphe non orienté G défini sur le même ensemble de sommets. Il apparaît lorsque l'on compare des voies métaboliques à des réseaux génomiques ou fonctionnels, et il est NP-difficile. Le modèle exact ILP2 demande quelques dizaines de secondes sur des instances de 100 sommets et plusieurs minutes sur des instances de 200 sommets, tandis que les décodeurs à base de réseaux de neurones sur graphes proposés auparavant renvoient des chemins d'environ un sommet. Notre méthode, Skew-GRAM, emprunte le modèle Skip-Gram au traitement automatique des langues (TAL) : les marches aléatoires orientées sur D sont lues comme des phrases, et les embeddings des sommets sont appris avec des exemples négatifs tirés parmi les non-successeurs. Un décodeur Monte-Carlo combine ces embeddings avec une anticipation exacte par programmation dynamique sur D et un bonus pour les sommets adjacents dans G au chemin en construction. Une dernière étape Union-Find extrait le plus long sous-chemin connexe dans G, si bien que tout chemin renvoyé est réalisable. Sur 368 instances d'Erdős–Rényi disposant d'une solution ILP2, avec cinq graines aléatoires, la méthode atteint la longueur ILP2 dans 71,3 % des exécutions à 100 sommets et dans 30,0 % à 200 sommets, avec des accélérations médianes par rapport à ILP2 de 4,0× et 21,2× ; sur une instance de 200 sommets, elle trouve un chemin plus long que la solution ILP2 fournie. Elle trouve l'optimum exact sur les 25 voies métaboliques KEGG et traite des graphes de 1000 sommets en 143 secondes environ. Sans l'anticipation par programmation dynamique, les chemins sont 23 % plus courts sur les instances de 100 sommets.

Mots-clés : Optimisation combinatoire · Bio-informatique · Embeddings de sommets · Skip-Gram · Programmation dynamique · Union-Find.

1 Introduction

Les systèmes biologiques se décrivent naturellement par des réseaux dont les sommets sont des entités biologiques et dont les arêtes sont des relations entre elles. Comparer deux réseaux construits sur les mêmes entités peut faire apparaître des motifs qu'aucun des deux réseaux ne montre seul. Le métabolisme, par exemple, est un réseau orienté dans lequel les réactions se succèdent le long des voies, tandis que la proximité génomique ou les interactions protéine–protéine donnent un réseau non orienté d'associations fonctionnelles [15, 18]. Une chaîne de réactions dont les gènes sont aussi proches les uns des autres sur le génome est candidate pour être une unité fonctionnelle conservée.

Le problème *One-to-one SkewGRAM* formalise cette recherche [1, 8]. Étant donnés un graphe orienté acyclique D et un graphe non orienté G sur le même ensemble de sommets, il s'agit de trouver un plus long chemin orienté de D dont les sommets induisent un sous-graphe connexe de G. Le problème est NP-difficile [3, 8]. Sa difficulté vient de l'interaction entre les deux contraintes : un successeur valide dans D peut rompre la connexité dans G, et des sommets proches dans G ne se trouvent pas forcément sur un même chemin de D.

Les méthodes exactes résolvent le problème à l'optimum. Le programme linéaire en nombres entiers ILP2 [1, 3] demande environ 30 secondes sur les instances de 100 sommets et environ quatre minutes sur les instances de 200 sommets de notre jeu de test. Des approches par apprentissage ont aussi été étudiées [2] : un double réseau convolutif sur graphes suivi d'un post-traitement heuristique, et un encodeur de graphe associé à un décodeur LSTM avec attention (DL2). Leur décodage séquentiel produit des chemins très courts, de 1,2 sommet en moyenne pour DL2.

Cet article suit une autre voie, du traitement automatique des langues (TAL) vers la biologie. Notre méthode, Skew-GRAM, lit les marches aléatoires orientées sur D comme des phrases et apprend des embeddings des sommets avec le modèle Skip-Gram [13], comme le font DeepWalk et node2vec pour les graphes [9,14]. Les embeddings seuls ne résolvent pas le problème d'optimisation : nous les utilisons comme l'un des trois signaux d'un décodeur conçu pour le problème, à côté d'une anticipation exacte par programmation dynamique sur D et d'un signal de connexité souple issu de G. Une étape d'extraction exacte impose ensuite la contrainte de connexité.

Nos contributions sont les suivantes :

1. un modèle d'embeddings Skip-Gram pour les sommets d'un DAG, entraîné sur des marches aléatoires orientées, avec sous-échantillonnage des hubs et échantillonnage négatif structurel ;
2. un décodeur guidé qui combine une anticipation exacte du plus long chemin dans D, la similarité des embeddings et un bonus de connexité dans G, suivi d'une extraction exacte par Union-Find qui garantit la réalisabilité ;
3. une évaluation sur 375 instances synthétiques de 100 et 200 sommets (dont 368 avec une solution ILP2) avec cinq graines aléatoires, sur 15 instances

plus grandes de 500 et 1000 sommets et sur 25 voies métaboliques réelles de KEGG [10] dont l'optimum exact est connu, ainsi qu'une ablation de l'anticipation.

2 Travaux connexes

Le problème One-to-one SkewGRAM appartient à une famille de problèmes de comparaison de réseaux biologiques hétérogènes, comme la recherche de motifs métaboliques et génomiques dans CoMetGeNe [18] et les méthodes de comparaison de réseaux présentées dans [15]. Fertin et al. [8] ont étudié sa complexité et proposé des algorithmes exacts. Ahmed Sidi et al. ont proposé les programmes linéaires en nombres entiers ILP1 et ILP2 [1] puis des modèles de programmation par contraintes [3] ; ILP2 est la référence exacte utilisée dans cet article.

Parmi les méthodes d'apprentissage pour l'optimisation combinatoire sur graphes figurent les réseaux de pointeurs [17], l'apprentissage par renforcement de politiques de construction [5, 6] et l'entraînement non supervisé de réseaux de neurones sur graphes [11]. Ces modèles sont entraînés sur de nombreuses instances, puis décodent une solution. Les approches d'apprentissage profond de [2] suivent ce schéma pour notre problème. Notre modèle est au contraire entraîné sur chaque instance séparément, car les identifiants des sommets n'ont pas de sens commun d'un graphe généré indépendamment à l'autre, et les embeddings appris ne font que guider une recherche dont la réalisabilité est garantie de façon exacte. Le modèle d'embeddings lui-même suit word2vec [13] et ses déclinaisons pour les graphes, DeepWalk [14] et node2vec [9].

3 Formulation du problème

Soit V un ensemble de n entités biologiques, $D = (V, A)$ un graphe orienté acyclique qui modélise une relation causale ou séquentielle, et $G = (V, E)$ un graphe non orienté qui modélise une association fonctionnelle. Un chemin $P = (p_1, \ldots, p_k)$ est dit (D, G)-*consistant* si

$$(p_i, p_{i+1}) \in A \quad (i = 1, \ldots, k-1) \qquad \text{et} \qquad G[\{p_1, \ldots, p_k\}] \text{ est connexe.} \quad (1)$$

L'objectif est de trouver un chemin (D, G)-consistant ayant le plus grand nombre de sommets. La figure 1 montre une petite instance.

La connexité ne peut pas être vérifiée de façon incrémentale. La contrainte de connexité est globale. Deux sommets consécutifs d'un chemin réalisable ne sont pas forcément adjacents dans G, et un préfixe d'un chemin réalisable peut être non connexe dans G alors que le chemin complet l'est (figure 1). C'est le cas général : dans notre jeu de test, les 368 solutions ILP2 contiennent toutes deux sommets consécutifs non adjacents dans G, et 363 d'entre elles (98,6 %) ont un préfixe non connexe. Un décodeur qui rejetterait toute extension rendant

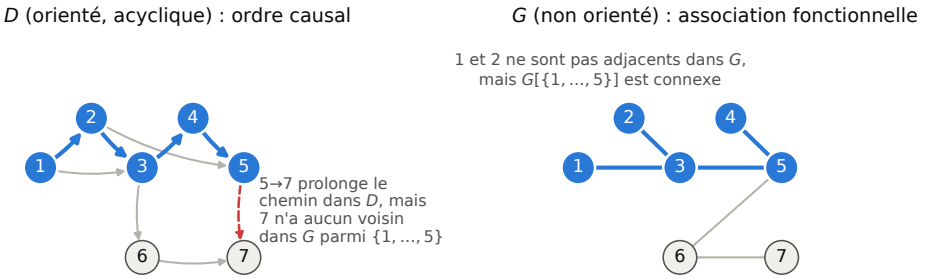


Fig. 1. Instance jouet à sept sommets. Le chemin mis en évidence $1 \to 2 \to 3 \to 4 \to 5$ est l'unique optimum. Le prolonger par l'arc $5 \to 7$ donne le plus long chemin de D, mais le sommet 7 n'a aucun voisin dans G parmi les autres sommets du chemin. Le préfixe $(1, 2)$ n'est pas connexe dans G, alors que le chemin complet l'est.

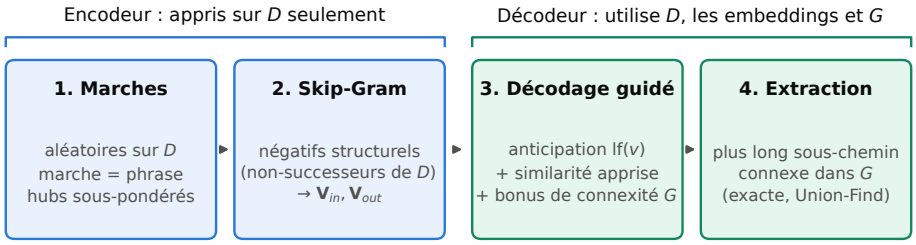


Fig. 2. Les quatre phases de Skew-GRAM, exécutées une fois par instance. L'encodeur apprend les embeddings $\mathbf{V}_{in}, \mathbf{V}_{out}$ à partir des marches sur D ; le décodeur génère des chemins candidats (étape A) et extrait de chacun un sous-chemin (D, G)-consistant (étape B).

le préfixe courant non connexe ne pourrait construire la plupart de ces solutions. Notre décodeur n'utilise donc G que comme un signal souple pendant la construction du chemin, et vérifie la connexité de façon exacte une fois le chemin terminé.

4 Méthode

Skew-GRAM comprend un encodeur, qui apprend les embeddings des sommets à partir de D seulement, et un décodeur, qui construit des chemins à partir de D, des embeddings et de G (figure 2). La méthode s'exécute une fois par instance, en quatre phases. L'analogie qui la guide vient du TAL : un sommet joue le rôle d'un mot, une marche aléatoire orientée celui d'une phrase, et le voisinage d'un sommet dans une marche celui de son contexte.

Phase 1 : marches aléatoires et sous-échantillonnage des hubs. Depuis chaque sommet ayant au moins un successeur, nous lançons plusieurs marches aléatoires qui suivent les arcs sortants de D : depuis le sommet u, le sommet suivant est tiré uniformément parmi ses successeurs $N^+(u)$. Une marche s'arrête sur un puits ou à une longueur maximale. Comme dans word2vec, chaque occurrence d'un sommet v de fréquence relative $f(v)$ est conservée avec la probabilité $\min\big(1, \sqrt{t/f(v)} + t/f(v)\big)$, $t = 10^{-3}$, ce qui réduit le poids des hubs. Une fenêtre glissante sur chaque marche produit ensuite les couples (centre, contexte).

Phase 2 : embeddings Skip-Gram avec négatifs structurels. Nous apprenons deux matrices $\mathbf{V}_{in}, \mathbf{V}_{out} \in \mathbb{R}^{n \times d}$. Pour un sommet central c, un sommet de contexte o et K sommets négatifs $n_1, \ldots, n_K$, la perte est l'objectif classique d'échantillonnage négatif

$$\mathcal{L}(c, o, n_{1:K}) = -\log \sigma(\mathbf{v}_{in,c}^\top \mathbf{v}_{out,o}) - \sum_{k=1}^{K} \log \sigma(-\mathbf{v}_{in,c}^\top \mathbf{v}_{out,n_k}). \tag{2}$$

La différence avec le Skip-Gram standard tient à la façon de tirer les négatifs : au lieu d'une distribution fondée sur les fréquences, les négatifs de c sont tirés uniformément parmi les sommets qui ne sont pas des successeurs de c dans D. Le signal négatif reflète ainsi l'orientation des arcs. Le décodeur utilise la similarité $\mathrm{sim}(u, v) = \mathbf{v}_{in,u}^\top \mathbf{v}_{out,v}$.

Phase 3 : décodage guidé (étape A). Un décodeur guidé par la seule similarité tend à choisir des successeurs qui mènent rapidement à un puits. Nous calculons donc, pour chaque sommet v, le nombre $\mathrm{lf}(v)$ d'arcs d'un plus long chemin de D partant de v :

$$\mathrm{lf}(v) = 0 \text{ si } N^+(v) = \emptyset, \qquad \mathrm{lf}(v) = \max_{w \in N^+(v)} \big(1 + \mathrm{lf}(w)\big) \text{ sinon.} \tag{3}$$

Comme D est acyclique, toutes les valeurs s'obtiennent en un seul parcours d'un ordre topologique inversé, en temps $O(|V| + |A|)$ [4]. Lorsque le sommet courant est u et que $v \in N^+(u)$ n'a pas encore été visité, le décodeur attribue à v le score

$$S(u, v) = \frac{\mathrm{lf}(v)}{\tau_{lf}} + \beta \, \frac{\mathrm{sim}(u, v)}{\tau} + \log(\gamma) \, \mathbb{I}_G(v), \tag{4}$$

où $\mathbb{I}_G(v) = 1$ si v est adjacent dans G à un sommet du chemin en construction, et 0 sinon. Le sommet suivant est tiré avec la probabilité $P(v \mid u) \propto \exp S(u, v)$. Les trois termes mesurent respectivement le potentiel de chemin restant dans D, la compatibilité apprise entre u et v, et une préférence pour les sommets reliés dans G au chemin déjà construit. Les sommets visités sont masqués. Le décodeur répète W fois ces marches depuis des sommets de départ tirés uniformément.

Phase 4 : extraction exacte (étape B). Un chemin candidat $P = (p_1, \ldots, p_L)$ issu de la phase 3 est un chemin de D, mais il peut ne pas être connexe dans G. Tout sous-chemin contigu de P reste un chemin de D ; il suffit donc de trouver le plus long intervalle $[i, j]$ tel que $G[\{p_i, \ldots, p_j\}]$ soit connexe. Pour chaque début i, nous étendons la fenêtre d'un sommet à la fois et maintenons les composantes de la fenêtre dans une structure Union-Find [16], en comparant chaque nouveau sommet aux sommets déjà présents dans la fenêtre. Le coût est en $O(L^3\alpha(L))$ pour un chemin de longueur L, où α est la réciproque de la fonction d'Ackermann. L'extraction n'est lancée que lorsque le candidat brut est plus long que le meilleur sous-chemin trouvé jusque-là, et L est borné par le plus long chemin de D (au plus 100 sommets dans nos instances). Le meilleur sous-chemin extrait sur les W marches est renvoyé ; il est (D, G)-consistant par construction.

5 Expériences

5.1 Protocole expérimental

Instances synthétiques. Le jeu de test principal contient 300 instances de $n = 100$ sommets et 75 instances de $n = 200$ sommets, chacune fournie avec la solution d'ILP2 (chemin et temps de calcul). D est construit en tirant un ordre topologique aléatoire puis en ajoutant chaque arc ń vers l'avant ż avec la probabilité p ($p = 0{,}08$ pour $n = 100$, $p = 0{,}05$ pour $n = 200$), et G est un graphe d'Erdős–Rényi $G(n, p)$ indépendant [7]. ILP2 ne renvoie aucune solution pour 2 instances de 100 sommets et 5 instances de 200 sommets ; elles sont exclues de toutes les comparaisons avec ILP2, ce qui laisse 298 et 70 instances. ILP2 est un modèle exact, mais les solutions fournies ne sont pas toutes optimales : sur l'instance 200_68, Skew-GRAM trouve un chemin réalisable de 10 sommets, que nous avons vérifié de façon indépendante, alors que le fichier ILP2 en indique 7. Nous utilisons donc les solutions ILP2 comme longueurs de référence, et nous disons qu'une exécution atteint ILP2 lorsque son chemin est au moins aussi long. Pour étudier le passage à l'échelle, nous utilisons aussi 10 instances de $n = 500$ sommets et 5 instances de $n = 1000$ sommets, construites de la même manière avec $p = 0{,}05$ (nous avons généré nous-mêmes les instances de 1000 sommets) ; aucune solution ILP2 n'est disponible pour elles.

Réseaux métaboliques réels. Nous utilisons 25 voies métaboliques de KEGG [10], issues de sept bactéries (codes KEGG aae, bsu, eco, mtv, pae, sau et syn). Dans chaque instance, D est le graphe métabolique d'une voie et G un graphe de proximité génomique ou fonctionnelle sur les mêmes sommets. L'ensemble contient 19 voies acycliques (au plus trois par organisme, avec 10 à 80 entités listées) et 6 voies dont les seuls cycles sont des réactions réversibles (2-cycles) ; les sommets sans aucun arc ni arête sont écartés au chargement, ce qui laisse de 4 à 38 sommets. Chaque réaction réversible est orientée de l'identifiant de sommet le plus petit vers le plus grand afin que D soit acyclique ; il s'agit

Tableau 1. Hyperparamètres, identiques pour toutes les instances.

Encodeur (phases 1–2)			Décodeur (phases 3–4)		
Marches par sommet		40	Marches de décodage	W	30 000
Longueur max. d'une marche		30	Température d'anticipation	τ_{lf}	1,5
Fenêtre de contexte		3	Température de similarité	τ	0,5
Dimension des embeddings	d	32	Poids de la similarité	β	0,3
Négatifs par couple	K	5	Bonus de connexité G	γ	6,0
Époques / taux d'appr.		8 / 0,01			

d'une simplification de modélisation. Ces instances n'ont pas de solution ILP2, mais elles sont petites : nous calculons donc leur optimum exact en énumérant les chemins de D avec deux coupes exactes. Les sommets d'un chemin réalisable appartiennent à une même composante connexe C de G, si bien que seuls les chemins de $D[C]$ sont énumérés, et un chemin qui se termine en u ne peut gagner plus de $\mathrm{lf}(u)$ sommets.

Déroulement. Le tableau 1 donne les hyperparamètres, identiques pour toutes les instances. L'entraînement utilise Adam [12]. Chaque instance est résolue avec cinq graines (42 à 46) ; la graine est réinitialisée au début de chaque instance, si bien qu'un résultat ne dépend pas de l'ordre de traitement des instances. Nous vérifions que chaque chemin renvoyé est (D, G)-consistant. Skew-GRAM tourne sur un processeur d'ordinateur portable Intel Core i5-1135G7 (4 cœurs), avec quatre instances résolues en parallèle sur un fil d'exécution chacune ; le portable servait aussi à d'autres tâches pendant les calculs, si bien que les temps comportent un bruit lié à la charge. Les temps d'ILP2 proviennent des fichiers du jeu de données et ont été mesurés sur une autre machine : les accélérations sont donc seulement indicatives.

5.2 Comparaison avec ILP2

Le tableau 2 résume les résultats. Les valeurs après ± sont les écarts-types, entre les cinq graines, de la moyenne correspondante, sauf pour les temps de calcul, où il s'agit de l'écart-type sur l'ensemble des exécutions.

À 100 sommets, les chemins trouvés par Skew-GRAM comptent 9,47 sommets en moyenne, contre 9,93 pour ILP2, soit un écart moyen de $-4{,}4\,\%$. La méthode atteint la longueur ILP2 dans 71,3 % des exécutions et en reste à au plus un sommet dans 90,3 %. Une exécution dure 6,9 s en moyenne, contre 29,1 s pour ILP2. À 200 sommets, le problème est plus difficile pour l'heuristique : l'écart moyen passe à $-17{,}2\,\%$ et la longueur ILP2 est atteinte dans 30,0 % des exécutions. ILP2 ralentit beaucoup plus, de 29,1 s à 251,8 s en moyenne, alors que Skew-GRAM passe de 6,9 s à 11,3 s : l'accélération médiane passe ainsi de 4,0× à 21,2× (figure 3b). À 100 sommets, aucune exécution ne renvoie un chemin plus long que celui d'ILP2 ; à 200 sommets, c'est le cas de quatre exécutions

Tableau 2. Skew-GRAM comparé à ILP2 sur les instances synthétiques disposant d'une solution ILP2 (cinq graines par instance). Exécutions atteignant ILP2 : exécutions dont le chemin est au moins aussi long que la solution ILP2. Accélération : médiane, sur les instances, du rapport entre le temps d'ILP2 et le temps moyen de Skew-GRAM.

	$n = 100$	$n = 200$
Instances avec une solution ILP2	298 / 300	70 / 75
ILP2		
Longueur moyenne (sommets)	9,93	9,64
Temps moyen (s)	29,1	251,8
Skew-GRAM		
Longueur moyenne (sommets)	$9{,}47 \pm 0{,}02$	$7{,}89 \pm 0{,}12$
Écart moyen à ILP2 (%)	$-4{,}4 \pm 0{,}2$	$-17{,}2 \pm 0{,}9$
Exécutions atteignant ILP2 (%)	$71{,}3 \pm 2{,}1$	$30{,}0 \pm 4{,}8$
Exécutions à 1 sommet au plus (%)	$90{,}3 \pm 1{,}0$	$53{,}1 \pm 3{,}3$
Temps moyen (s)	$6{,}9 \pm 2{,}6$	$11{,}3 \pm 4{,}2$
Accélération médiane / ILP2	4,0×	21,2×

sur cinq pour l'instance 200_68 (tableau 3). Pour mémoire, DL2 [2] trouvait des chemins de 1,2 sommet en moyenne sur des instances de 50 sommets ; nous ne l'avons pas relancé, ce chiffre n'est donc qu'indicatif.

Le tableau 3 détaille dix instances, choisies aux 0e, 50e, 80e, 95e et 100e centiles de l'écart moyen pour chaque taille. À 100 sommets, la plupart des instances atteignent la longueur ILP2, et l'écart se concentre sur quelques instances pour lesquelles les marches guidées atteignent rarement la région de D qui contient le meilleur chemin. À 200 sommets, les écarts sont plus grands (jusqu'à 8 sommets pour une exécution), tandis que sur l'instance 200_68 Skew-GRAM dépasse la solution ILP2 dans quatre exécutions sur cinq.

Stabilité selon la graine. Le décodeur est stochastique ; nous mesurons donc la variation des résultats selon la graine. La longueur moyenne sur l'ensemble des instances est très stable (écart-type de 0,02 sommet pour $n = 100$ et de 0,12 pour $n = 200$). Au niveau de chaque instance, la longueur renvoyée varie avec la graine de 0,43 sommet en moyenne (écart-type) pour $n = 100$ et de 0,93 pour $n = 200$; les cinq exécutions renvoient la même longueur pour 48 % et 23 % des instances. Le temps de calcul d'une instance varie lui aussi d'une exécution à l'autre (écart-type de 2,4 s en moyenne pour $n = 100$), surtout parce que la machine servait à d'autres tâches : la quantité de calcul est la même à chaque exécution, puisque le nombre de marches est fixé. Garder la meilleure des cinq exécutions porterait le nombre d'instances où la longueur ILP2 est atteinte à 273 sur 298 et 42 sur 70, pour un coût cinq fois plus élevé.

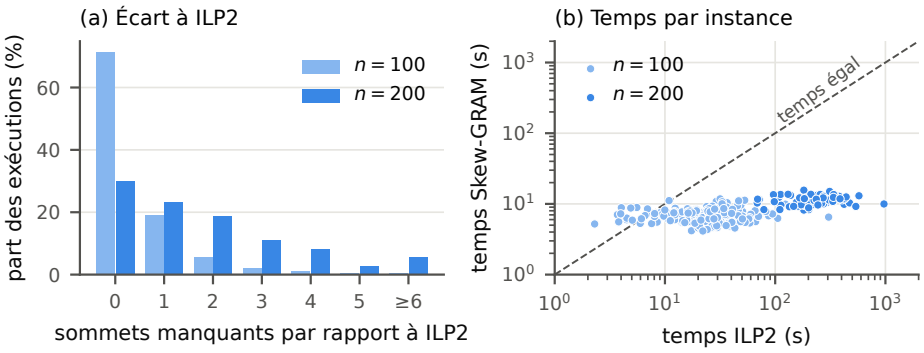


Fig. 3. (a) Nombre de sommets manquant au chemin de Skew-GRAM par rapport à la solution ILP2, sur l'ensemble des exécutions (0 : longueur ILP2 atteinte ou dépassée). (b) Temps de calcul par instance (Skew-GRAM : moyenne sur les cinq graines) ; les points sous la droite en pointillés sont plus rapides avec Skew-GRAM.

5.3 Réseaux métaboliques réels

Le tableau 4 présente les résultats sur les 25 voies KEGG. Skew-GRAM trouve l'optimum exact sur chaque voie et avec chaque graine. Ces instances sont bien plus faciles que les instances synthétiques : elles ont au plus 38 sommets et G est très creux (8,7 arêtes en moyenne), si bien que l'optimum est court (3,4 sommets en moyenne) et se réduit à un seul sommet pour 7 voies. L'énumération exacte avec la coupe par composantes prend moins de 2 ms par voie, bien moins que les 2,9 s de Skew-GRAM. Sur des réseaux aussi petits, une méthode exacte est le meilleur choix ; l'expérience montre que la méthode s'applique telle quelle à des données réelles et renvoie des solutions correctes, mais elle ne démontre pas d'avantage en vitesse. Des réseaux réels plus grands, comme des réseaux métaboliques à l'échelle du génome couplés à des données d'interactions protéiques, sont nécessaires pour évaluer la méthode là où les solveurs exacts peinent.

5.4 Ablation de l'anticipation

Pour mesurer le rôle de l'anticipation, nous retirons le terme $\mathrm{lf}(v)/\tau_{lf}$ de $S(u, v)$ en gardant tout le reste inchangé (graine 42). À 100 sommets, la longueur moyenne passe de 9,49 à 7,29 sommets ($-23{,}2\,\%$), les chemins raccourcissent sur 229 des 298 instances, et la longueur ILP2 n'est plus atteinte que sur 47 instances au lieu de 216. À 200 sommets, la longueur moyenne passe de 7,87 à 6,40 ($-18{,}7\,\%$) et la longueur ILP2 n'est atteinte que sur 5 instances au lieu de 24. La similarité des embeddings ne dit pas au décodeur si un successeur mène à une longue branche ou à un puits ; l'anticipation fournit cette information de façon exacte, pour un coût linéaire.

Tableau 3. Résultats par instance pour dix instances, choisies aux 0e, 50e, 80e, 95e et 100e centiles de l'écart moyen pour chaque taille (de la meilleure à la moins bonne). Skew-GRAM : moyenne ± écart-type sur les cinq graines. Un écart positif signifie un chemin plus long que la solution ILP2.

	Longueur (sommets)		Écart	Temps (s)	
Instance	ILP2	Skew-GRAM	(%)	ILP2	Skew-GRAM
100_4	11	11,0 ± 0,0	0,0	29,0	6,1
100_230	10	9,8 ± 0,4	−2,0	16,1	8,2
100_133	13	12,0 ± 0,0	−7,7	20,6	6,4
100_247	6	5,0 ± 0,0	−16,7	27,0	9,3
100_300	9	5,0 ± 0,0	−44,4	16,9	7,9
200_68	7	9,2 ± 1,3	31,4	183,9	11,9
200_50	9	7,6 ± 1,3	−15,6	134,6	9,1
200_61	7	5,0 ± 0,0	−28,6	440,3	12,5
200_49	9	5,4 ± 0,9	−40,0	225,2	11,2
200_41	13	7,2 ± 1,6	−44,6	108,1	12,4

Tableau 4. Résultats sur 25 voies métaboliques KEGG (cinq graines chacune). LP(D) : nombre de sommets d'un plus long chemin de D. Optimum : optimum exact par énumération. Atteint : voies sur lesquelles Skew-GRAM atteint l'optimum (moyenne sur les graines). Les longueurs sont des moyennes.

Voies	$\lvert V\rvert$	$\lvert A\rvert$	$\lvert E\rvert$	LP(D)	Optimum	Skew-GRAM	Atteint
Acycliques (19)	4–15	25,9	9,1	7,5	3,58	3,58	19/19
Avec 2-cycles (6)	12–38	97,3	7,3	13,5	2,83	2,83	6/6
Toutes (25)	4–38	43,1	8,7	9,0	3,40	3,40	25/25

5.5 Passage à l'échelle

Le tableau 5 montre comment le temps de calcul croît avec n et où il est dépensé. La génération des marches est négligeable. L'entraînement croît avec la taille du corpus de marches, qui augmente avec n et avec la longueur des marches. Le décodage représente la plus grande part du temps à toutes les tailles, de 91 % du temps pour $n = 100$ à 57 % pour $n = 1000$: son coût est proportionnel au nombre de marches W, à la longueur de chaque marche, bornée par le plus long chemin de D, et au nombre de successeurs évalués à chaque pas, et le bonus G compare chaque candidat aux sommets déjà présents dans le chemin. Avec $n = 1000$ (environ 25 000 arcs), une exécution prend 143 s en moyenne. Aucune solution ILP2 n'est disponible pour ces tailles, mais le nombre de sommets d'un plus long chemin de D, LP(D), est une borne supérieure de l'optimum : les chemins trouvés comptent 33,5 et 72,8 sommets en moyenne pour $n = 500$ et $n = 1000$, contre 46,6 et 94,4 pour cette borne.

Les 30 000 marches représentent le coût principal ; nous examinons donc comment la qualité dépend de W. Comme les k premières marches d'une exécution

Tableau 5. Temps de calcul par phase et longueur des chemins selon n (toutes les instances, cinq graines). $|A|$ et $\mathrm{LP}(D)$ (sommets d'un plus long chemin de D) sont des moyennes sur les instances ; Longueur : moyenne ± écart-type, entre graines, de la moyenne ; Total : moyenne ± écart-type sur les exécutions.

				Longueur	Temps par exécution (s)			
n	Inst.	$\lvert A\rvert$	$\mathrm{LP}(D)$	(sommets)	Marches	Entraîn.	Décodage	Total
100	300	399	15,1	$9{,}47 \pm 0{,}02$	0,02	0,6	6,2	$6{,}8 \pm 2{,}6$
200	75	999	19,6	$7{,}83 \pm 0{,}15$	0,06	2,6	8,6	$11{,}3 \pm 4{,}2$
500	10	6185	46,6	$33{,}52 \pm 1{,}32$	0,25	19,1	29,7	$49{,}1 \pm 8{,}6$
1000	5	24974	94,4	$72{,}76 \pm 1{,}82$	0,58	60,5	81,4	$142{,}6 \pm 13{,}2$

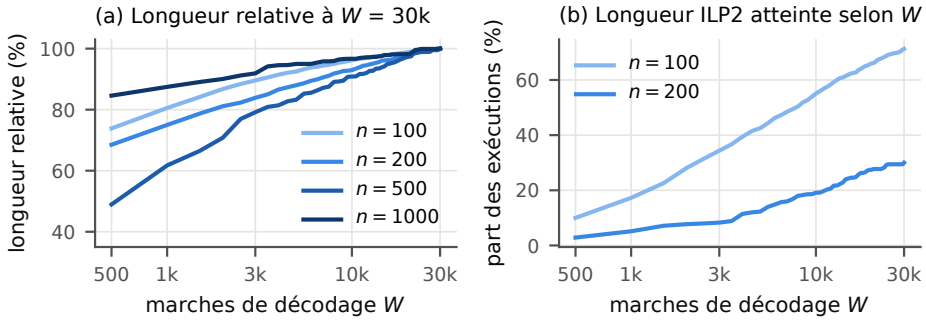


Fig. 4. Effet du nombre de marches de décodage W. (a) Longueur moyenne des chemins, relative à la longueur obtenue avec $W = 30\,000$. (b) Part des exécutions qui atteignent la longueur ILP2 (instances disposant d'une solution ILP2).

utilisent exactement les mêmes nombres aléatoires qu'une exécution avec un budget $W = k$, la meilleure longueur après chaque bloc de 500 marches donne, sans calcul supplémentaire, le résultat de chaque budget plus petit (figure 4). La qualité augmente encore à $W = 30\,000$, pour toutes les tailles. Avec $W = 3\,000$, les chemins conservent entre 79 % ($n = 500$) et 92 % ($n = 1000$) de leur longueur finale, et la part des exécutions qui atteignent la longueur ILP2 tombe de 71,3 % à 34,4 % pour $n = 100$. Réduire W n'est donc pas une économie gratuite. Trois pistes découlent de la structure du décodeur. Les marches sont indépendantes et peuvent s'exécuter en parallèle. Le bonus G peut être maintenu de façon incrémentale en conservant l'ensemble des voisins dans G du chemin courant, ce qui le rend en $O(1)$ par candidat au lieu de $O(L)$. Enfin, le budget pourrait s'adapter à l'instance, par exemple en s'arrêtant lorsque la meilleure longueur n'a pas progressé depuis un nombre donné de marches. Nous laissons ces améliorations d'implémentation à des travaux futurs.

5.6 Discussion et limites

Les résultats confirment l'intérêt de combiner des composants appris et exacts. L'anticipation fournit au décodeur une information exacte, peu coûteuse à calculer sur un DAG, au lieu de demander au modèle d'embeddings de l'apprendre. G sert au guidage et à la vérification finale plutôt qu'à l'objectif d'apprentissage des embeddings, ce qui recentre l'apprentissage sur l'ordre du chemin, tandis que l'étape d'extraction garantit la réalisabilité.

La méthode a des limites claires. C'est une heuristique, qui ne doit pas remplacer ILP2 lorsque l'optimalité doit être garantie. Son écart à ILP2 croît avec n à budget fixé, et sur de petites voies réelles l'énumération exacte est à la fois exacte et plus rapide. Les embeddings sont entraînés pour chaque instance, ce qui ajoute un coût d'entraînement qu'un modèle entraîné sur plusieurs instances éviterait. Enfin, notre évaluation sur données réelles ne couvre que de petites voies, et l'orientation des réactions réversibles est une simplification.

6 Conclusion

Nous avons présenté Skew-GRAM, qui applique le modèle Skip-Gram du TAL au problème One-to-one SkewGRAM issu de la biologie : trouver un plus long chemin dans un DAG dont les sommets sont connexes dans un second graphe. Des embeddings appris à partir de marches aléatoires orientées guident un décodeur Monte-Carlo, avec une anticipation exacte sur le DAG et un bonus de connexité, et une étape exacte d'Union-Find rend tout chemin renvoyé réalisable. Sur 368 instances synthétiques disposant d'une solution ILP2, la méthode atteint la longueur ILP2 dans la plupart des exécutions à 100 sommets et s'exécute bien plus vite qu'ILP2, avec une accélération qui croît avec la taille des graphes ; ses résultats moyens varient peu d'une graine à l'autre, elle trouve l'optimum exact sur 25 voies métaboliques réelles, et elle passe à l'échelle de graphes de 1000 sommets. Nos travaux futurs porteront sur des réseaux réels plus grands, un décodeur plus rapide (marches en parallèle, bonus G incrémental, budget adaptatif) et des embeddings entraînés sur plusieurs instances.

Références

1. Ahmed Sidi, M.L., Bocquillon, R., Mohamed Babou, H., Dhib, C., Néron, E., Soukhal, A., Nanne, M.F. : Improved approaches to solve the one-to-one SkewGraM problem. Computers & Operations Research **138**, 105584 (2022)
2. Ahmed Sidi, M.L., Mohamed Babou, H., Dhib, C. : Deep learning approaches for finding a longest DG-consistent path in biological networks. Preprint (2024)
3. Ahmed Sidi, M.L., Mohamed Babou, H., Dhib, C., Néron, E., Soukhal, A., Nanne, M.F. : Constraint programming approaches for finding conserved metabolic and genomic patterns. Computers & Operations Research **183**, 107166 (2025)
4. Bellman, R. : On a routing problem. Quarterly of Applied Mathematics **16**(1), 87–90 (1958)

5. Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S. : Neural combinatorial optimization with reinforcement learning. arXiv preprint arXiv :1611.09940 (2016)
6. Dai, H., Khalil, E.B., Zhang, Y., Dilkina, B., Song, L. : Learning combinatorial optimization algorithms over graphs. In : Advances in Neural Information Processing Systems, vol. 30 (2017)
7. Erdős, P., Rényi, A. : On random graphs I. Publicationes Mathematicae Debrecen **6**, 290–297 (1959)
8. Fertin, G., Mohamed Babou, H., Rusu, I. : Algorithms for subnetwork mining in heterogeneous networks. In : Experimental Algorithms (SEA 2012). LNCS, vol. 7276, pp. 184–194. Springer (2012)
9. Grover, A., Leskovec, J. : node2vec : scalable feature learning for networks. In : Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 855–864 (2016)
10. Kanehisa, M., Goto, S. : KEGG : Kyoto Encyclopedia of Genes and Genomes. Nucleic Acids Research **28**(1), 27–30 (2000)
11. Karalias, N., Loukas, A. : Erdős goes neural : an unsupervised learning framework for combinatorial optimization on graphs. In : Advances in Neural Information Processing Systems, vol. 33 (2020)
12. Kingma, D.P., Ba, J. : Adam : a method for stochastic optimization. arXiv preprint arXiv :1412.6980 (2014)
13. Mikolov, T., Sutskever, I., Chen, K., Corrado, G., Dean, J. : Distributed representations of words and phrases and their compositionality. In : Advances in Neural Information Processing Systems, vol. 26 (2013)
14. Perozzi, B., Al-Rfou, R., Skiena, S. : DeepWalk : online learning of social representations. In : Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 701–710 (2014)
15. Sharan, R., Ideker, T. : Modeling cellular machinery through biological network comparison. Nature Biotechnology **24**(4), 427–433 (2006)
16. Tarjan, R.E. : Efficiency of a good but not linear set union algorithm. Journal of the ACM **22**(2), 215–225 (1975)
17. Vinyals, O., Fortunato, M., Jaitly, N. : Pointer networks. In : Advances in Neural Information Processing Systems, vol. 28 (2015)
18. Zaharia, A., Labedan, B., Froidevaux, C., Denise, A. : CoMetGeNe : mining conserved neighborhood patterns in metabolic and genomic contexts. BMC Bioinformatics **20**, 19 (2019)